Practice Test 3: Answers and Explanations

PRACTICE TEST 3 ANSWER KEY

1. D
2. A
3. C
4. C
5. D
6. B
7. A
8. B
9. C
10. D
11. A
12. A
13. D
14. C
15. B
16. B
17. A
18. D
19. C
20. B
21. B
22. C
23. D
24. D
25. A
26. B
27. B
28. D
29. B
30. A
31. B
32. B
33. D
34. B
35. D
36. D
37. D
38. A
39. D
40. B
41. A
42. D

ANSWER EXPLANATIONS

Section I: Multiple-Choice Questions

1. **D** This question tests how well you understand assigning values to variables and following the steps of the code. When `trial()` is called, `a` is assigned to the integer value of 10 and `b` is assigned to the integer value of 5. Next, `doubleValues()` is called with `a` and `b` as inputs `c` and `d`. The method `doubleValues()` multiplies the input values `c` and `d` by 2 and prints them out. Thus, the value of `c` is 10 * 2 = 20 and the value of `d` is 5 * 2 = 10. Because `System.out.print()` does not print any line breaks or spaces, the resulting print is `2010`. While `c` and `d` have been reassigned new values, these values exist only within the `doubleValues()` call, so the values of `a` and `b` are unchanged. After the `doubleValues()` method is completed, the `trial()` method then prints the values of `b` and then `a`, printing `510`. Once again, no spaces or line breaks are printed, so, combined together, what is printed from calling `trial()` is `2010510`. Therefore, the correct answer is (D).

2. **A** The method `mystery(int a, int b)` takes as input integers `a` and `b` with the precondition that `a > b > 0`. Plug in `x = 5` and `y = 2`. These values are passed as parameters to make `a = 5` and `b = 2`. Set `d = 0`. Now go to the `for` loop, initializing `c = a = 5`. Since it is still the case that `c > b`, execute the `for` loop. Execute `d = d + c` by setting `d = 0 + 5 = 5`. Decrease `c` by 1 to get `c = 4`. Since `c > b`, execute the `for` loop again. Execute `d = d + c` by setting `d = 5 + 4 = 9`. Decrease `c` by 1 to get `c = 3`. Since it is still the case that `c > b`, execute the `for` loop again. Execute `d = d + c` by setting `d = 9 + 3 = 12`. Decrease `c` by 1 to get `c = 2`. Since it is no longer the case that `c > b`, stop executing the `for` loop. Return `d = 12`. Now go through the choices and eliminate any that don't describe a return of 12. Choice (A) is the sum of all integers greater than `y` but less than or equal to `x`. The sum of all integers greater than 2 but less than or equal to 5 is 3 + 4 + 5 = 12, so keep (A). Choice (B) is the sum of all integers greater than or equal to `y` but less than or equal to `x`. The sum of all integers greater than or equal to 2 but less than or equal to 5 is 2 + 3 + 4 + 5 = 14, so eliminate (B). Choice (C) is the sum of all integers greater than `y` but less than `x`. The sum of all integers greater than 2 but less than 5 is 3 + 4 = 7, so eliminate (C). Choice (D) is the sum of all integers greater than or equal to `y` but less than `x`. The sum of all integers greater than or equal to 2 but less than 5 is 2 + 3 + 4 = 9, so eliminate (D). Only one answer remains. The correct answer is (A).

3. **C** The question asks for what is printed by the call `mystery(3)`. In the call, 3 is taken as a parameter and assigned to `n`. The integer `k` is then initialized in the `for` loop and set equal to 0. Since `k < n`, execute the `for` loop. Call `mystery(0)`, while keeping your place in the call of `mystery(3)`. Again, `k` is initialized in the `for` loop and set equal to 0. In this call, since `n = 0`, it is not the case that `k < n`, so do not execute the `for` loop. This completes the call of `mystery(0)`, so return to `mystery(3)`. The next command is `System.out.print(k)`. Since `k = 0`, the first character printed is `0`. Look to see whether any choices can be eliminated. All choices begin with `0`, so no choices can be eliminated. Continue with the method call. Since this is the last command of the `for`

loop, increment k to get k = 1. Since it is still the case that k < n, execute the for loop again. Call mystery(1), again keeping your place in the call of mystery(3). Again, k is initialized in the for loop and set equal to 0. In this call, since n = 1, it is the case that k < n, so execute the for loop. Call mystery(k), which is mystery(0). As seen above, mystery(0) prints nothing, so execute the next line in mystery(1), which is System.out.print(k). Since k = 0, print 0. Eliminate (A), since the second character printed is not 0. Increment k to get k = 1. Since it is no longer the case that k < n, do not execute the for loop again and end this call of mystery(1). Return to the original call of mystery(3), where k = 1 and System.out.print(k) is the next statement. Print 1. All remaining choices have 1 as the third character, so do not eliminate any choices. This is the end of the for loop, so increment k to get k = 2. Since n = 3, it is still the case that k < n, so execute the for loop. Call mystery(2) while holding your place in mystery(3). Again, k is initialized in the for loop and set equal to 0. In this call, since n = 2, it is the case that k < n, so execute the for loop. Call mystery(0), which prints nothing. Execute System.out.print(k) to print 0. Eliminate (B), since the next character is not 0. Increment k to get k = 1. Since k < n, execute the for loop. Call mystery(1), which, as described above, prints 0. All remaining choices have 0 as the next character, so don't eliminate any choices. The next command in mystery(2) is System.out.print(k), so print 1. Again, keep all the remaining choices. In mystery(2), increment k to get k = 2. Since n = 2, it is no longer the case that k < n, so stop executing the for loop. End mystery(2) and return to mystery(3), where k = 2 and the next command is System.out.print(k), so print 2. Again, keep all remaining choices. Increment k to get k = 3. Since it is no longer the case that k < n, stop executing the for loop. The method call is complete for mystery(3), so there will be no more printed characters. Eliminate (D), which has further characters printed.

In other words, the calling sequence looks like this:

Function Call	Prints
`mystery(3)`	
`  mystery(0)`	
`  System.out.print(0)`	0
`  mystery(1)`	
`    mystery(0)`	
`    System.out.print(0)`	0
`  System.out.print(1)`	1
`  mystery(2)`	
`    mystery(0)`	
`    System.out.print(0)`	0
`    mystery(1)`	
`      mystery(0)`	
`      System.out.print(0)`	0
`    System.out.print(1)`	1
`  System.out.print(2)`	2

The correct answer is (C).

4. **C** `SelectionSort` walks through the array to find the smallest element in the part of the array not yet sorted. It then swaps that smallest element with the first unsorted element.

Start with the original array of values.

```
4    10    1    2    6    7    3    5
```

The smallest element is 1. Swap that with the first unsorted element, 4. The array now looks like this.

```
1    10    4    2    6    7    3    5
```

This is not a choice, so continue. The smallest element in the unsorted part of the array (from 10 to 5) is 2. Swap that with the first unsorted element, 10.

```
1    2    4    10    6    7    3    5
```

This is still not a choice, so continue this process. The smallest element in the unsorted part of the array (from 4 to 5) is 3. Swap that with the first unsorted element, 4.

```
1    2    3    10    6    7    4    5
```

This is (C), so stop here. If you're interested, here are the complete moves for selection sort.

```
1    2    3    4    6    7    10    5
1    2    3    4    5    7    10    6
1    2    3    4    5    6    10    7
1    2    3    4    5    6    7    10
```

The correct answer is (C).

5. **D** Execute the commands. The integer `k` and the integer array `A` are initialized. The array `A` is given length 7. Thus, the array, `A`, has seven elements with indexes from 0 to 6. Execute the first `for` loop. Set `k` = 0. Since `k < A.length`, execute the `for` loop, which sets `A[0]` equal to `A.length - k` = 7 – 0 = 7. Increment `k` to get `k` = 1. Since 1 < 7, execute the `for` loop, which sets `A[1]` equal to `A.length - k` = 7 – 1 = 6. Increment `k` to get `k` = 2. Since 2 < 7, execute the `for` loop, which sets `A[2]` equal to `A.length - k` = 7 – 2 = 5. Increment `k` to get `k` = 3. Since 3 < 7, execute the `for` loop, which sets `A[3]` equal to `A.length - k` = 7 – 3 = 4. Increment `k` to get `k` = 4. Since 4 < 7, execute the `for` loop, which sets `A[4]` equal to `A.length - k` = 7 – 4 = 3. Increment `k` to get `k` = 5. Since 5 < 7, execute the `for` loop, which sets `A[5]` equal to `A.length - k` = 7 – 5 = 2. Increment `k` to get `k` = 6. Since 6 < 7, execute the `for` loop, which sets `A[6]` equal to `A.length - k` = 7 – 6 = 1. Increment `k` to get `k` = 7. Since it is not the case that 7 < 7, stop executing the `for` loop. Therefore, the array has the values

```
7    6    5    4    3    2    1
```

The second `for` loop puts the value of the `k`th element of the array into the `(k + 1)`st element. Be careful!

A quick glance might lead you to believe that array elements `A[1]` to `A[5]` are shifted one spot to the right, giving an answer of (C). However, a step-by-step analysis demonstrates that this will not be the case. The `for` loop sets `k` = 0. Since 0 < `A.length` – 1, execute the `for` loop. The command in the `for` loop is `A[k + 1] = A[k]`. Since `k` = 0, set `A[1] = A[0]` = 7. Increment `k` to get `k` = 1. Since it is still the case that `k` < `A.length` – 1, execute the `for` loop. Since `k` = 1, set `A[2] = A[1]` = 7. Continue in this manner. Since `A.length` – 1 = 7 – 1 = 6, only execute the `for` loop through `k` = 5. Each execution of the `for` loop is shown below.

k	Assignment	A[]
0	A[1] = A[0]	7 7 5 4 3 2 1
1	A[2] = A[1]	7 7 7 4 3 2 1
2	A[3] = A[2]	7 7 7 7 3 2 1
3	A[4] = A[3]	7 7 7 7 7 2 1
4	A[5] = A[4]	7 7 7 7 7 7 1
5	A[6] = A[5]	7 7 7 7 7 7 7

The final values contained by `A` are `7 7 7 7 7 7 7`. The correct answer is (D).

6. **B** Choices (C) and (D) are syntactically invalid according to the given class definitions. In (C), `p` is used as a `PostOffice` object rather than an array of `PostOffice` objects. Choice (D) treats `getMail` and `getBox` as static methods without invoking them from an object.

Choices (A) and (B) differ only in the indexes of the array and methods. There are two clues in the question that indicate that (B) is the correct answer. First, `p` is declared as an array of 10 `PostOffices`. This means that `p[10]` would raise an `ArrayListOutOfBoundsException`. Remember that `p[9]` actually refers to the 10th post office, since array indexes begin with 0. Second, the comments in the class definitions for the `getBox` and `getMail` methods indicate that the parameter they take is zero-based. Therefore, they should be passed an integer one less than the number of the box or piece of mail needed. For either reason, eliminate (A). The correct answer is (B).

7. **A** In the method `printEmptyBoxes`, the loop variable `k` refers to the index of the post office and the loop variable `x` refers to the index of the box within the post office. Choice (A) is correct. It checks to see whether the box is assigned and whether it does not have mail using the appropriate methods of the `Box` class. It then prints out the box number of the box. Choice (B) is similar to (A) but incorrectly interchanges `x` and `k`. Eliminate (B). Choice (C) omits the call to the method `getBox`. Therefore, it attempts to call the `getBoxNumber()` method of the `PostOffice` object `p[k]`. Since `PostOffice` objects do not have a `getBoxNumber()` method, this would result in a compile error. Eliminate (C). Choice (D) is similar to (C), but interchanges `x` and `k`. Eliminate (D). The correct answer is (A).

8. **B** There are four possibilities for the values of `a` and `b`. Either both `a` and `b` are true, both `a` and `b` are false, `a` is true and `b` is false, or `a` is false and `b` is true.

Analyze the possibilities in a table.

a (initial value)	b (initial value)	a = a && b (final value)	b = a || b (final value)
true	true	true	true
true	false	false	false
false	true	false	true
false	false	false	false

Remember when calculating `b = a || b` that `a` has already been modified. Therefore, use the final, rather than the initial, value of `a` when calculating the final value of `b`.

Go through each statement. Statement I says that the final value of `a` is equal to the initial value of `a`. This is not the case in the second row of the table, so Statement I is not always true. Eliminate any choice that includes it: (A) and (C). Statement II says that the final value of `b` is equal to the initial value of `b`. This is true in each row and, thus, Statement II is always true. Statement III says that the final value of `a` is equal to the initial value of `b`. This is not the case in the third row of the table, so Statement III is not always true. Eliminate any choice that includes it: (D). The correct answer is (B).

9. **C** The best way to solve this problem is to look at each `if` statement individually. The integer `x` is initialized and then set equal to 53. The next statement is an `if` statement with the condition `(x > 10)`. Since 53 > 10, execute the statement, `System.out.print("A")`, and print `A`. The next statement is an `if-else` statement, this one having the condition `(x > 30)`. Since 53 > 30, execute the `if` statement, `System.out.print("B")`, and print `B`. Even though 53 > 40, because the next part of the statement is an `else` statement, it cannot be executed if the original `if` statement was executed. Therefore, do not execute the `else` statement, and do not print `C`. The next statement is another `if` statement, this one having the condition `(x > 50)`. Since 53 > 50, execute the statement, `System.out.print("D")`, and print `D`. The next statement is another `if` statement, this one having the condition `(x > 70)`. Since it is not the case that 53 > 70, do not execute the statement. There is no further code to execute, so the result of the printing is `ABD`. The correct answer is (C).

10. **D** This problem tests your ability to work with nested `for` loops. Although it appears complicated at first glance, it can easily be solved by systematically walking through the code.

Be sure to write the values of the variables `j` and `k` down on paper; don't try to keep track of `j` and `k` in your head. Use the empty space in the question booklet for this purpose.

Code	j	k	Output
Set `j` to the value `-2`.	-2		
Test the condition: `j <= 2`? Yes.	-2		
Set `k` to the value that `j` has.	-2	-2	
Test the condition: `k < j + 3`? Yes.	-2	-2	
Output `k`.	-2	-2	`-2`
Update k: `k++`	-2	-1	`-2`
Test the condition: `k < j + 3`? Yes.	-2	-1	`-2`
Output `k`.	-2	-1	`-2 -1`
Update k: `k++`	-2	0	`-2 -1`
Test the condition: `k < j + 3`? Yes.	-2	0	`-2 -1`
Output `k`.	-2	0	`-2 -1 0`
Update k: `k++`	-2	1	`-2 -1 0`
Test the condition: `k < j + 3`? No.	-2	1	`-2 -1 0`
Update j: `j = j + 2`	0	1	`-2 -1 0`
Test the condition: `j <= 2`? Yes.	0	1	`-2 -1 0`
Set `k` to the value that `j` has.	0	0	`-2 -1 0`
Test the condition: `k < j + 3`? Yes.	0	0	`-2 -1 0`
Output `k`.	0	0	`-2 -1 0 0`

At this point, stop because (D) is the only choice that starts `-2 -1 0 0`. However, repeating this process will result in the correct output of `-2 -1 0 0 1 2 2 3 4`. The correct answer is (D).

11. **A** To solve this problem, first note that `count % 3` is equal to the remainder when `count` is divided by 3. Execute the method call `mystery(5, "X")`, taking `count = 5` and `s = "X"` as parameters. Because it is not the case that `5 <= 0`, do not execute the first `if` statement.

Because 5 % 3 = 2, calling `mystery(5, "X")` will execute the `else` statement, printing `X`. Then, it will call `mystery(4, "X")`, and return. Because 4 % 3 = 1, calling `mystery(4, "X")` will execute the `else if` statement, printing `X-X`, call `mystery(3, "X")`, and return. At this point, (C) can be eliminated. Because 3 % 3 = 0, calling `mystery(3, "X")` will execute the `if` statement, printing `X--X`, call `mystery(2, "X")`, and return. Note that you can stop at this point because (B) and (D) can be eliminated. If you're interested, here is the remainder of the execution. Because 2 % 3 = 2, calling `mystery(2, "X")` will execute the `else` statement, printing `X`, call `mystery(1, "X")`, and return. Because 1 % 3 = 1, calling `mystery(1, "X")` will execute the `else`

`if` statement, printing `X-X`, call `mystery(0, "X")`, and return. Finally, calling `mystery(0, "X")` will simply return because `count` is less than or equal to zero. Putting it all together, `mystery(5, "X")` prints

```
XX-XX--XXX-X
```

The correct answer is (A).

12. **A** The only information that you need to solve this problem is the first sentence in the description of the constructor. Class `DiningRoomSet` has a constructor, which is passed a `Table` object and an `ArrayList` of `Chair` objects. Because you are writing a constructor, you can immediately eliminate (B), which is a `void` method. Constructors never return anything, so there is never a need to specify a `void` return. Choice (D) is incorrect because the constructor is passed a `Table` object and a `Chair` object, not a `Table` object and an `ArrayList` of `Chair` objects. Choice (C) is incorrect because the second parameter has two types associated with it. It should have only one type: `ArrayList`. Choice (A) is correct.

13. **D** The best way to solve this problem is to eliminate choices. Choices (A) and (B) are incorrect because the class description states that the `getPrice` method of the `DiningRoomSet` class does not take any parameters. Choice (C) can be eliminated because the private data field `myChairs` is not a `Chair`; it is an `ArrayList`. Therefore, it does not have a `getPrice` method. This leaves (D). You need to know that `ArrayLists` are accessed using the `get` method while arrays are accessed using the `[]` notation. `MyChairs` is an `ArrayList`, so the correct answer is (D).

14. **C** To solve this type of problem, use information about the output and the loops to eliminate incorrect choices. Then, if necessary, work through the code for any remaining choices to determine the correct answer. There are six lines of output. By examining the outer loop of each of the choices, you can eliminate (B) because it will traverse the loop seven times, and during each traversal at least one number will be printed. You can also eliminate (A) because the outer loop will never be executed. The initial value of j is 6, but the condition `j < 0` causes the loop to terminate immediately. The remaining choices have the same outer loop, so turn your attention to the inner loop. Finally, eliminate (D) because the condition of the inner loop, `k >= 0`, will cause a 0 to be printed at the end of each line. This leaves (C), which is indeed the correct answer.

15. **B** This problem tests your knowledge of the methods of the `ArrayList` class. The following table shows the contents of `list` after each line of code.

Code	Contents of list	Explanation
`list = new ArrayList();`	`[]`	A newly created `ArrayList` is empty
`list.add(new Integer(7));`	`[7]`	Adds 7 to the end of `list`
`list.add(new Integer(6));`	`[7, 6]`	Adds 6 to the end of `list`
`list.add(1, new Integer(5));`	`[7, 5, 6]`	Inserts 5 into `list` as position 1, shifting elements to the right as necessary
`list.add(1, new Integer(4));`	`[7, 4, 5, 6]`	Inserts 4 into `list` as position 1, shifting elements to the right as necessary
`list.add(new Integer(3));`	`[7, 4, 5, 6, 3]`	Adds 3 to the end of `list`
`list.set(2, new Integer(2));`	`[7, 4, 2, 6, 3]`	Replaces the number at position 2 in `list` with 2
`list.add(1, new Integer(1));`	`[7, 1, 4, 2, 6, 3]`	Inserts 1 into `list` at position 1, shifting elements to the right as necessary

The next command is to print `list`. Therefore, the correct answer is (B).

16. **B** `IOException` must be in the header of the method that creates the `File`, in this example the `main` method. The correct answer is (B).

17. **A** The object bob is an instance of the `Fish` class. When `bob.action()` is called, it will execute the `action()` method in the `Fish` class. Thus `"splash splash"` is printed. The correct answer is (A).

18. **D** The insertion sort algorithm creates a sorted array by sorting elements one at a time.

The `for` loop of the code shows that the elements are being sorted from left to right. To determine the position of the new element to be sorted, the value of the new element must be compared with the values of the sorted elements from right to left. This requires `index--` in the `while` loop, not `index++`. Choices (A) and (C) are wrong.

In order to place the new element to be sorted in its correct position, any sorted elements larger than the new element must be shifted to the right. This requires `sort[index] = sort[index -1]`. Choice (B) is wrong. The correct answer is (D).

19. **C** In order for the array to be sorted in descending order, you will need to make a change. Plug each answer choice into the array to see which is correct. In (A) and (B), as the index will never be less than 0, the contents of the `while` loop will never be executed. Choice (C) is the correct answer, as it changes the condition of finding the position of the new element from being less than the com-

pared element to greater. In (D), altering the `for` loop this way would lead to the elements being sorted from right to left but still in ascending order. The correct answer is (C).

20. **B** The rate at which insertion sort runs depends on the number of comparisons that are made. The number of comparisons is minimized with an array with elements that are already sorted in ascending order and maximized with an array with elements that are sorted in descending order. The array in (B) is sorted in reverse order and will require the most comparisons to sort. The correct answer is (B).

21. **B** All three responses look very similar, so look carefully at the difference. Statement I is missing the keyword `new`, which is needed to create a new object. Eliminate any choice that includes Statement I: (C). Statements II and III differ in that Statement II uses `f.numerator` and `f.denominator` while Statement III uses `f.getNumerator()` and `f.getDenominator()`. Since the integers `numerator` and `denominator` are private, while the methods `getNumerator()` and `getDenominator()` are public, the methods must be called by another object. Therefore, Statement II is not valid. Eliminate any choice that includes it: (A) and (D). The correct answer is (B).

22. **C** Go through the choices one at a time. Choice (A) is incorrect because the multiplication operator, `*`, is not defined to work on `Fraction` objects. Eliminate (A). Choice (B) is incorrect because the multiply method takes only one parameter and it is not correctly invoked by a `Fraction` object. Eliminate (B). Choice (C) correctly calls the `multiply()` method as through a fraction object, taking the other fraction as a parameter. Keep (C). Finally, while (D) calculates the value of the result of the multiplication, the `"/"` operator assigns integer types, which cannot be applied to `answer`, which is an object of type `Fraction`. Eliminate (D). The correct answer is (C).

23. **D** Go through each statement one at a time. Constructor I is legal. This default constructor of the `ReducedFraction` class will automatically call the default constructor of the `Fraction` class and the private data of both classes will be set appropriately. Eliminate any choice that does not include Constructor I: (B) and (C). Because Constructor II is not included in the remaining choices, do not worry about analyzing it. Constructor III is also legal. The call `super(n, d)` invokes the second constructor of the `Fraction` class, which sets the private data of the `Fraction` class appropriately. The private data of the `ReducedFraction` class is then set explicitly in the `ReducedFraction` constructor. Note that if the call `super(n, d)` were not present, Constructor III would still be legal. However, it would create a logical error in the code, as the default constructor of the `Fraction` class would be invoked, and the private data of the `Fraction` class would not be set appropriately. Eliminate (A), which does not include Constructor III. Only one choice remains, so there is no need to continue. However, to see why Constructor II is illegal, remember that derived classes may not access the private data of their super classes. In other words, the constructor for a `ReducedFraction` may not directly access `numerator` and `denominator` in the `Fraction` class. The correct answer is (D).

24. **D** Go through each statement one at a time. Statement I is incorrect. "==" checks object references as opposed to their contents. This is an important distinction as two different objects may hold the same data. Eliminate (A) and (C), which contain Statement I. Both of the remaining choices contain Statement II, so don't worry about analyzing it. Statement III is correct. The `compareTo()` method of the `String` class returns 0 if the two `String` objects hold the same strings. Eliminate (B), which does not include Statement III. Only one choice remains, so there is no need to continue. However, you should see why Statement II is also correct. The `equals` method of the `String` class returns true if the two `String` objects hold the same strings. The correct answer is (D).

25. **A** The values of `s` and `t` are not changed in `mystery()`. Even though `s` and `t` are both parameters of the method, only the instance variables `a` and `b` are changed. Thus, the original Strings `s` and `t` are unaffected by any action in `mystery()`. The correct answer is (A).

26. **B** There are certain characters in Java that have meaning to the Java language. The double quotes character, ", and the backslash, \ are among those characters. In order to print one of these "special" characters, they must be preceded by a backslash. In summary, to print the character ", a programmer must use \". To print the character \, a programmer must use \\. All other characters in the string were "normal" in that they do not need anything special to print. Choice (B) is the correct answer.

27. **B** An example will help clarify this question.

Consider the following `for` loop:

```
for (int k = 0; k < 3; k++)
{
      System.out.println(k);
}
```

This prints out the integers 0 to 2, one number per line. Matching `int k = 0` to <1>; `k < 3` to <2>; `k++` to <3> and `System.out.println(k);` to <4>, go through each choice one at a time and determine whether each has the same functionality.

Choice (A) initializes `k` and assigns it the value 0. Then it tests to determine whether `k < 3`. This is true, so execute the `while` loop. The next command is `k++`, so increase `k` by 1 to get `k = 0 + 1 = 1`. Now execute `System.out.println(k)` to print `1`. However, the first number printed in the original was `0`, so this is incorrect. Eliminate (A).

Choice (B) initializes `k` and assigns it the value 0. Then it tests to determine whether `k < 3`. This is true, so execute the `while` loop. Now execute `System.out.println(k)` to print `0`. The next command is `k++`, so increase `k` by 1 to get `k = 0 + 1 = 1`. Go back to the top of the `while` loop. Since `k = 1`, it is still the case that `k < 3`, so execute the `while` loop. Now execute `System.out.println(k)` to print `1`. The next command is `k++`, so increase `k` by 1 to get `k = 1 + 1 = 2`. Go back to the top of the `while` loop. Since `k = 2`, it is still the case that `k < 3`, so execute the `while` loop.

Now execute `System.out.println(k)` to print 2. The next command is `k++`, so increase `k` by 1 to get `k = 2 + 1 = 3`. Go back to the top of the `while` loop. Since `k = 3`, it is no longer the case that `k < 3`, so stop executing the `while` loop. The result of the program is printing the integers 0 to 2, one number per line, so keep (B).

Choice (C) initializes `k` and assigns it the value 0. Then it tests the condition `!(k < 3)`. Since it is the case that `k < 3`, the statement `(k < 3)` has the boolean value `true`, making the boolean value of `!(k < 3)` `false`. Thus the `while` loop is not executed. Since the `while` loop is not executed, nothing is printed. Eliminate (C).

Choice (D) initializes `k` and assigns it the value 0. The next command is `k++`, so increase `k` by 1 to get `k = 0 + 1 = 1`. Then it tests to determine whether `k < 3`. This is true, so execute the `while` loop. Execute `System.out.println(k)` to print 1. However, the first number printed in the original was 0, so this is incorrect. Eliminate (D).

The correct answer is (B).

28. **D** This question tests your knowledge of operator precedence. In Java, multiplication, division, and modulus are performed before addition and subtraction. If more than one operator in an expression has the same precedence, the operations are performed left to right. Parenthesizing the expression one step at a time:

```
      a / b + c - d % e * f
→     (a / b) + c - d % e * f
→     (a / b) + c - (d % e) * f
→     (a / b) + c - ((d % e) * f)
→     ((a / b) + c) - ((d % e) * f)
```

The correct answer is (D).

29. **B** Come up with a sample array of 10 strings with length 5. Let `String[] x` be `{"gator", "teeth", "ducky", "quack", "doggy", "woofs", "kitty", "meows", "bears", "growl"}`. The code must print the first letter of all 10 strings, followed by the second letter of all 10 strings, and so on. Therefore, it must print

```
gtdqdwkmbgaeuuooieer...
```

Go through each segment one at a time.

Segment I initializes `i` and `j` with no values. The outer `for` loop sets `i = 0`, and the inner `for` loop sets `j = 0`. The instruction of the inner `for` loop is `System.out.print(x[i].substring(j, j + 1)`. The `substring(a, b)` method of the `String` class returns a string made up of all the characters starting with the index of `a` through the index of `b - 1`. Therefore, `x[i].substring(j, j + 1)` returns the characters of `x[i]` from index `j` through index `(j + 1) - 1`. Since `(j + 1) - 1 = j`, it returns all the characters from indexes `j` through `j`—a single character string made up of the character at index `j`. Therefore, if `i = 0` and `j = 0`, `System.out.print(x[i].substring(j, j + 1)`

returns the character at index 0 of the string at index 0. This is the "g" from "gator". This is what should be printed, so continue. The inner `for` loop increments `j` to get `j = 1`. Since `j < 5`, the loop is executed again, printing the character at index 1 of the string at index 0. This is the character "a" from "gator". However, the character "t" from "teeth" should be the next character printed, so Segment I does not execute as intended. There is no need to continue with I, but note that it will eventually print all of the characters of the first string followed by all of the characters of the second string, and so on. Eliminate (A) and (C), which include I.

Both remaining choices include II, so don't worry about checking this one. Instead, analyze III.

Segment III initializes `i` and `j` with no values. The outer `for` loop sets `i = 0`, and the inner `for` loop sets `j = 0`. The instruction of the inner `for` loop is `System.out.print(x[i].substring(j, j + 1)`. As discussed above, when `i = 0` and `j = 0`, this command returns the character at index 0 of the string at index 0, which is the "g" from "gator". This is what should be printed, so continue. The inner `for` loop increments `j` to get `j = 1`. Since `j < 5`, the loop is executed again, printing the character at index 1 of the string at index 0. This is the character "a" from "gator". Again, the character "t" from "teeth" should be the next character printed, so Segment III does not execute as intended. There is no need to continue with Segment III, but note that it will attempt to print the first 10 characters of the first 5 strings. Since each String contains only 5 characters, an `IndexOutOfBoundsException` will be thrown when `j = 5` and the program attempts to print the character at index 5. Eliminate (D), which includes Segment III.

Only one answer remains, so there is no need to continue. However, to see why Segment II is correct, note that it is similar to Segment I, but it reverses the roles of `i` and `j`. The command `System.out.print(x[j].substring(i, i + 1))` prints the character at index `j` of the string at index `i`. The inner `for` loop increments the index of the `String` array before the `outer` loop increments the index of the character in each `String`. Therefore, Segment II correctly prints out the first character of all 10 strings followed by the second character of all 10 strings, and so on. The correct answer is (B).

30. **A** Begin with (D) and determine whether an error would occur. It is perfectly legal to assign a value of type `int` to a variable of type `double` in an expression, so eliminate (D). However, certain rules apply when evaluating the expression. In the expression `c = a / b;` `a` and `b` are both integers. Therefore, the `/` operator represents integer division. The result of the division is truncated by discarding the fractional component. In this example, 7 / 4 has the value 1—the result of truncating 1.75. When assigning an integer to a variable of type `double`, the integer value is converted to its equivalent double value. Therefore, the correct answer is (A).

31. **B** For the code to print the word `Yes` two conditions must be true. The remainder must be zero when `x` is divided by 2. In other words, `x` must be divisible by 2: that is, even. The value after truncating the result when `x` is divided by 3 must be 1. In other words, $1 \leq x / 3 < 2$. The second condition is more narrow than the first. The only integers that fulfill the second condition are 3, 4, and 5. Of those, only 4 is even and therefore also fulfills the first condition. Therefore, the correct answer is (B).

32. **B** Go through each statement one at a time. Segment I is incorrect because all parameters in Java are passed by value. After a method returns to its caller, the value of the caller's parameters is not modified. The strings will not be swapped. Eliminate (A) and (C), which include Segment I. Since both remaining choices include Segment III, don't worry about it. Segment II is incorrect because `myName` is a private data field of the `SomeClass` class and may not be accessed by the `swap` method. Note that the question specifically states that the `swap` method is not a method of the `SomeClass` class. Eliminate (D), which includes Segment II. Only one choice remains, so there is no need to continue. To see that Segment III correctly swaps the names of the objects, note that it calls the public methods `setName()` and `getName()` rather than the private `String myName`. Because the references of the instance object variables are the same as the references of the class object variables, modifying the data of the instance variables also changes the data of the class variable. The correct answer is (B).

33. **D** This loop is an infinite loop. It never reaches the condition where `a` is no longer less than `b`. Set up a table and walk through a few iterations of the loop. The if statement is looking to see if `b` is evenly divisible by `a`, so we will follow that if statement to determine which part of the if statement is executed.

Iteration	Initial value of a	Initial value of b	Statements executed	Final value of a	Final value of b

1	2	6	`if (b % a == 0)`
			true so
			`a++;`
			`b--;`
				3	5

2	3	5	`if (b % a == 0)`
			false so
			`b++;`
			`a--;`
				2	6

3	2	6	`if (b % a == 0)`
			true so
			`a++;`
			`b--;`
				3	5

As you can quickly see, `a` and `b` are going to alternate between two sets of values and will never reach the point where `a` will no longer be less than `b`. This is an infinite loop and (D) is the correct answer.

34. **B** In order to solve this recursive problem, work backward from the base case to the known value of `mystery(4)`.

Let y represent the <missing value>. Note that `mystery(1) = y`. Now calculate `mystery(2)`, `mystery(3)`, and `mystery(4)` in terms of y.

```
mystery(2) = 2 * mystery(1) + 2
           = 2 * y + 2
mystery(3) = 2 * mystery(2) + 3
           = 2 * (2 * y + 2) + 3
           = 4 * y + 4 + 3
           = 4 * y + 7
mystery(4) = 2 * mystery(3) + 4
           = 2 * (4 * y + 7) + 4
           = 8 * y + 14 + 4
           = 8 * y + 18
```

Because `mystery(4)` also equals 34, set 8 * y + 18 = 34 and solve for y.

8 * y + 18 = 34
8 * y = 16
y = 2

The correct answer is (B).

35. **D** The `for` loop terminates when the condition is no longer true. This can happen either because `k` is no longer less than `X.length` or because `X[k]` does not equal `Y[k]`. Choice (D) states this formally. Another way to approach the problem is to use De Morgan's Law to negate the condition in the `for` loop. Recall that De Morgan's Law states that

`!(p && q)` is equivalent to `!p || !q`

Negating the condition in the `for` loop gives

```
   !(k < X.length && X[k] == Y[k])
=> !(k < X.length) || !(X[k] == Y[k])
=> k >= X.length || X[k] != Y[k]
```

This method also gives (D) as the correct answer.

36. **D** Choices (A), (B), and (C) are examples of an `ArithmeticException`, a `nullPointerException`, and an `ArrayIndexOutOfBoundsException`, respectively. Be careful! While (D) may appear to be an example of an `IllegalArgumentException`, it is actually an example of an error that is caught at compile time rather than at runtime. An `IllegalArgumentException` occurs when a method is called with an argument that is either illegal or inappropriate—for instance, passing –1 to a method that expects to be passed only positive integers. Therefore, the correct answer is (D).

37. **D** First note that `a` and `b` are of type `Double`, not of type `double`. This distinction is important. The type `double` is a primitive type; the type `Double` is a subclass of the `Object` class that implements the `Comparable` interface. A `Double` is an object wrapper for a `double` value. Go through each choice one at a time. Choice (A) is incorrect. It compares `a` and `b` directly rather than the `double` values inside the objects. Even if `a` and `b` held the same value, they might be different

objects. Eliminate (A). Choice (B) is incorrect, because `a.doubleValue()` returns a double. Since `double` is a primitive type, it does not have any methods. Had this choice been `!(a.equals(b))`, it would have been correct.

The expression `a.compareTo(b)` returns a value less than zero if `a.doubleValue()` is less than `b.doubleValue()`, a value equal to zero if `a.doubleValue()` is equal to `b.doubleValue()`, and a value greater than zero if `a.doubleValue()` is greater than `b.doubleValue()`. Choice (C) is incorrect because the `compareTo` method returns an int rather than a boolean. Choice (D) is correct. Since `compareTo()` returns `0` if and only if the two objects hold the same values, it will not return `0` if the values are different. The correct answer is (D).

38. **A** While "encapsulating functionality in a class" sounds like (and is) a good thing, "declaring all data fields to be public" is the exact opposite of good programming practice. Data fields to a class should be declared to be private in order to hide the underlying representation of an object. This, in turn, helps increase system reliability. Choices (B), (C), and (D) all describe effective ways to ensure reliability in a program. The correct answer is (A).

39. **D** Go through each method one at a time. Method 1 examines at most 10 words using a binary search technique on 1,024 pages to find the correct page. Remember, the maximum number of searches using binary search is $\log_2(n)$ where n is the number of entries. It then searches sequentially on the page to find the correct word. If the target word is the last word on the page, this method will examine all 50 words on the page. Therefore, Method 1 will examine at most 60 words. Eliminate (A) and (B) which don't have 60 for Method 1. The two remaining choices both have the same number for Method 2, so worry only about Method 3. Method 3 sequentially searches through all of the words in the dictionary. Because there are 51,200 words in the dictionary and the target word may be the last word in the dictionary, this method may have to examine all 51,200 words in order to find the target word. Eliminate (C), which does not have 51,200 for Method 3. Only one choice remains, so there is no need to continue. However, note that Method 2 first uses a sequential search technique to find the correct page. If the target word is on the last page, this method will examine the first word on all 1,024 pages. Then, as with Method 1, it may examine as many as all 50 words on the page to find the target word. Therefore, Method 2 will examine at most 1,074 words. The correct answer is (D).

40. **B** Pick a positive value of `j`. The question says that `j` is a positive integer. A recursive method is easiest if it takes fewer recursions to get to the base case, so try `j = 1`. If `j = 1`, then `m = 2j = 2(1) = 2`. Determine the return of `mystery(2)`. Since it is not the case that `m = 0`, the `if` condition is false, so execute the `else` statement, which is to return `4 + mystery(m - 2)`. Since `m - 2 = 2 - 2 = 0`, `mystery(2)` returns `4 + mystery(0)`. Determine the return of `mystery(0)`. In `mystery(0)`, `m = 0`, so the `if` condition is true, which causes the method to return 0. Therefore, `mystery(0)` returns 0, and `mystery(2)` returns `4 + mystery(0) = 4 + 0 = 4`. Go through each choice and eliminate any that are not 4. Choice (A) is `m`, which is 2, so eliminate (A). Choice (B) is `2m`, which is `2(2) = 4`, so keep (B). Choice (C) is `j`, which is 1, so eliminate (C). Choice (D) is `2j`, which is `2(1) = 2`, so eliminate (D). Only one choice remains. The correct answer is (B).

41. **A** The correct answer is easily found using a truth table.

	a	b	(a >= b) || (a != -b)	Final value
(A)	–5	5	(–5 >= 5) || (–5 != –5) F || F	F
(B)	5	–5	(5 >= –5) || (5 != – –5) T || F	T
(C)	–4	5	(–4>= 5) || (–4 != –5) F || T	T
(D)	–5	4	(–5 >= 4) || (–5 != –4) F || T	T

Choice (A) is the only answer that results in a false evaluation. Choice (A) is the correct answer.

42. **D** Eliminate choices (A) and (B) because they only call recur with one parameter and it needs two parameters. Next trace through the code using the other choices. Let's use `n` = 2 and `p` = 3. $2^3 = 8$ so we know the method should return 8.

We can assume that choice (D) is the answer. If we call `recur(2,3), p ==1` the base case, has not been met (p=3). So, execute return `n * recur(n, p-1)` or `2 * recur (2,2)`. Note at this point, the p did not change. The variable p will never change and never reach the base case. Eliminate (C).

Call `recur (n,p)`	Has base case been reached? `if (p==1)`	Return n or `n * recur(n, p-1)`	Substitution begins working from the bottom up
call `recur(2,3)`	No	`2 * recur (2, 2)`	2 * 4 = 8 final answer
`2 * recur (2, 2)`	No	`2 * recur (2, 1)`	2 * 2 = 4
`2 * recur (2, 1)`	Yes	2	

Choice (D) is the correct answer.

Section II: Free-Response Questions

1.

a.

```
public boolean findEmployeeForChild(Child c)
{
for (Employee e : employees)
{
   if (e.childrenAssigned() < maxRatio && e.canTeach(c.getAge()))
   {
        e.assignChild(c);
        return true;
   }
}

return false;
}
```

This can also be done with a typical `for` loop or a `while` loop and the `.get` method. A loop is necessary to look at each `Employee` in the `ArrayList`. You must then make sure the chosen `Employee` doesn't already have the maximum number of children allowed. You must also send the age of the `Child` using the `getAge` accessor to the `canTeach` method to see whether the chosen `Employee` is eligible to teach the given `Child`.

If an `Employee` is found for the given `Child`, you need to assign the `Child` to the `Employee` using the `assignChild` method and return `true`.

You should not return `false` inside the loop, since it is possible a different `Employee` is eligible to teach the given `Child`.

b.

```
public boolean addChild(Child c)
{
   if (findEmployeeForChild(c) == true)
   {
        children.add(c);
        return true;
   }

   return false;
 }
}
```

The solution must call the `findEmployeeForChild` method from part (a) to see whether there is an `Employee` eligible to teach the given `Child`. If there is, you add the `Child` to the `ArrayList` using the `add` method and return `true`. If there isn't, you return `false`.

`DayCare` Rubric

Part (a)

+5

+1	`for` loop correctly traverses all elements of the `employees` list
+1	`if` statement correctly determines whether the number of children assigned to the current `employee` is less than the maximum allowed
+1	`if` statement correctly determines whether current employee can teach the given child based on age
+1	The child is assigned to the employee if both conditions are met
+1	`true` is returned if the child is assigned to an employee; `false` is returned otherwise

Part (b)

+2

+1	The `findEmployeeForChild` method is called correctly
+1	Children are added to the `children` list correctly if an employee is found

2. Employee—Canonical Solution

```
public class Employee {
  private String name;
  private String role;
  private int salary;

  Employee (String n, String r, int s)
  {
     name = n;
     role = r;
     salary = s;
  }
  Employee (String n, int s)
  {
     name = n;
     role = "";
     salary = s;
  }
  public void setRole(String r) {
     role = r;
  }
  public void setSalary(int s)
  {
     salary = s;
  }
  public String toString()
  {
     String str = name + "\nposition is ";
     if (role.equals(""))
          str += "unknown";
     else
          str += role;
     str += "\nsalary is " + salary;
     return  str;
  }
}
```

`Employee` Rubric

+7

+1	public class `Employee`
+1	3 variables must be private, 2 strings, 1 int
+1	Constructors headers are correct: first constructor uses String, String, int as parameters, second constructor uses String, int as parameters
+1	Constructors initialize all 3 variables. Second constructor initializes salary to 0.
+1	Header for `changeRole` is correct
+1	Header for `toString` method is correct
+1	`toString` method returns string correctly using \n to skip to new lines and uses unknown if role is the empty string

3. `fullName`—Canonical Solution

a.
```
public static String getLastName(String name)
{
  int space = name.indexOf(" ");
  String last = name.substring(space + 1);
  return last;
}
```

The solution to part (b) is similar to that of part (a). You still need to find the location of the space, but the substring starts at the location of the space plus `1`, which will be the first letter of the last name. You could add a second parameter of `name.length()`, but it isn't required.

b.
```
public static int countVowels(String name)
{
  int count = 0;
  for (int i = 0; i < name.length(); i++)
  {
      String letter = name.substring(i, i + 1);
      if (letter.equals("a") || letter.equals("e") || letter.equals("i") ||
letter.equals("o") || letter.equals("u"))
          count++;
  }
  return count;
}
```

You need to create a loop to go through the entire name. You could use an `enhanced-for` loop to extract characters, but characters are not part of the AP subset. If you use them, make sure you use them correctly. With a traditional `for` loop, you need to extract each letter using the `substring` method and see whether it equals one of the vowels. A count variable is increased by `1` each time a vowel is found.

`fullName` Rubric

Part (a)

+2

+1	The `indexOf` method is used correctly to find the first space
+1	The `substring` method is used correctly to extract and return the last name

Part (b)

+3

+1	A loop is used to examine each letter in the name
+1	An `if` statement is used to determine whether the current letter is a vowel
+1	The correct vowel count is returned

4. `ParkingLot`—Canonical Solution

```
public boolean parkCar(Car newCar)
{
  if (openSpaces() > 0)
  {
      for (int r = 0; r < lot.length; r++)
      {
          for (int c = 0; c < lot[r].length; c++)
          {
              if (lot[r][c] == null)
              {
                  lot[r][c] = newCar;
                  return true;
              }
          }
      }
  }

  return false;
}
```

You need to use nested `for` loops to iterate through the `lot` 2D array. You could also use `lot[0].length` in the second `for` loop that iterates through the columns instead of `lot[r].length`. If you find a spot that is `null`, you need to set that spot to `newCar` and return `true`, in that order. There should be a `return false` at the end of the method or in an `else` statement. On the AP exam, if a question tells you to use a method (such as `openSpaces`) and you don't use it, you can lose points.

`ParkingLot` Rubric

+6

+1	The `openSpaces` method is called correctly to determine whether there are spaces available
+1	Nested `for` loops are used correctly to iterate through the `lot` 2D array
+1	An `if` statement checks whether the current spot in the array is `null`
+1	The `newCar` is assigned correctly to a `null` element in the 2D array
+1	The `newCar` is only assigned to one space
+1	The method returns `true` if a spot was found, and `false` otherwise

HOW TO SCORE PRACTICE TEST 3

Section I: Multiple-Choice

_____ × 1.964 = _____

Number Correct (out of 42) → Weighted Section I Score (Do not round)

Section II: Free-Response

Question 1: _____ (out of 7) × 2.700 = _____ (Do not round)

Question 2: _____ (out of 7) × 2.700 = _____ (Do not round)

Question 3: _____ (out of 5) × 2.700 = _____ (Do not round)

Question 4: _____ (out of 6) × 2.700 = _____ (Do not round)

Sum = _____

Weighted Section II Score (Do not round)

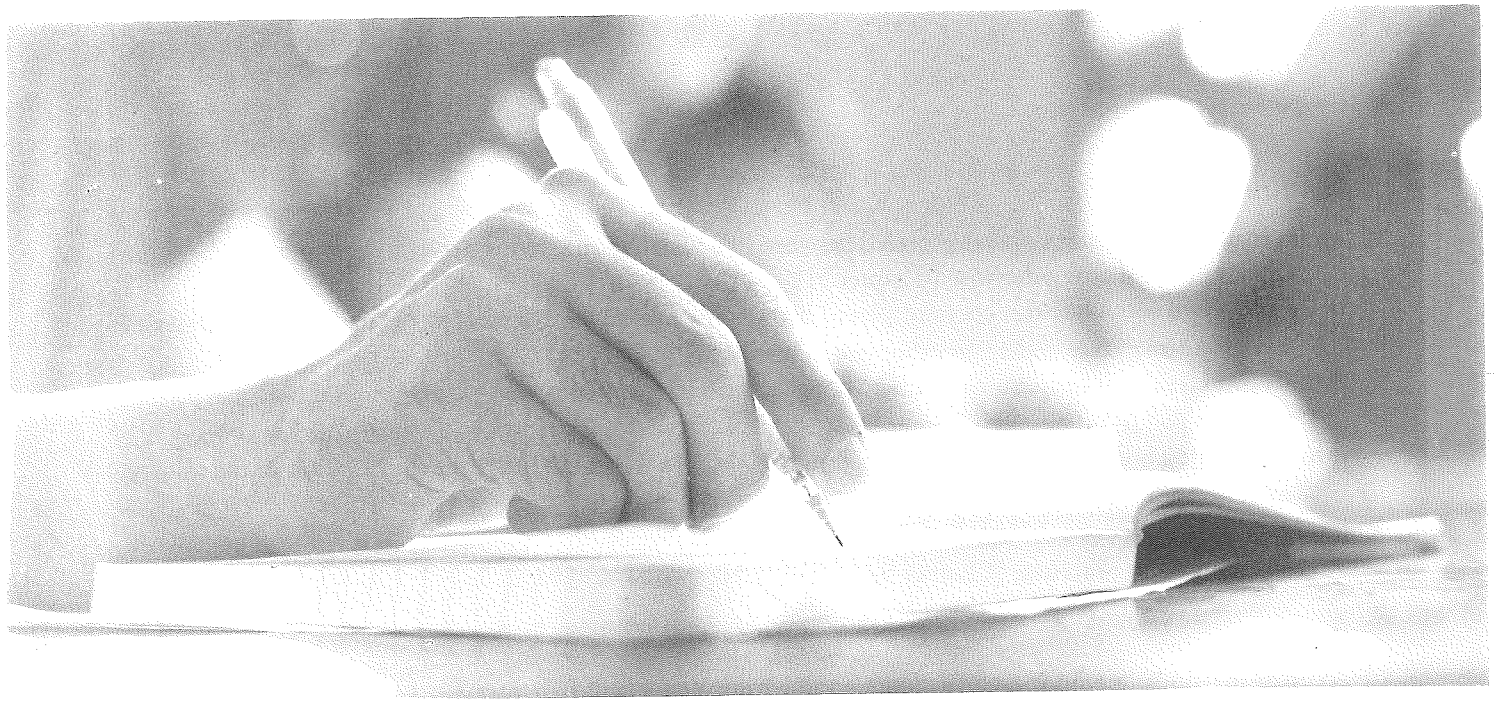
AP Score Conversion Chart Computer Science A

Composite Score Range	AP Score
107–150	5
90–106	4
73–89	3
56–72	2
0–55	1

Composite Score

_____ + _____ = _____

Weighted Section I Score + Weighted Section II Score = Composite Score (Round to nearest whole number)



Glossary

Symbols

== (is equal to): a boolean operator placed between two variables that returns true if the two values have the same value and false otherwise [Chapter 5]

!= (is not equal to): a boolean operator placed between two variables that returns `false` if the two variables have the same value and `true` otherwise [Chapter 5]

&& (logical *and*): an operator placed between two boolean statements that returns `true` if both statements are true and `false` otherwise [Chapter 5]

! (logical *not*): a boolean operator placed before a boolean statement that returns the negation of the statement's value [Chapter 5]

|| (logical *or*): an operator placed between two boolean statements that returns `false` if both statements are false and `true` otherwise [Chapter 5]

A

accessor methods: methods used to obtain a value from a data field [Chapter 7]

aggregate class: a class made up of, among other data, instances of other classes [Chapter 7]

array: an ordered list of elements of the same data type [Chapter 8]

`ArrayIndexOutOfBoundsException`: a run-time error caused by all calling of an index array that is either negative or greater than the highest index of the array [Chapter 8]

`ArrayList` object: an object of a type that is a subclass of `List` used on the AP Computer Science A Exam [Chapter 9]

assign: create an identifier by matching a data value to it [Chapter 3]

assignment operator: a single equals sign ("=") indicating that an identifier should be assigned a particular value [Chapter 3]

B

base case: the indication that a recursive method should stop executing and return to each prior recursive call [Chapter 11]

binary: a system of 1s and 0s [Chapter 3]

binary search: a search of a sorted array that uses searches beginning in the middle and determines in which direction to sort [Chapter 8]

blocking: a series of statements grouped by { } that indicate a group of statements to be executed when a given condition is satisfied [Chapter 5]

boolean: a primitive data type that can have the value `true` or `false` [Chapter 3]

boolean operator ==: an operator that returns `true` if it is between two variables with the same value and `false` otherwise [Chapter 5]

C

casting: forcing a data type to be recognized by the compiler as another data type [Chapter 3]

character: a primitive data type representing any single text symbol, such as a letter, digit, space, or hyphen [Chapter 3]

class: a group of statements, including control structures, assembled into a single unit [Chapters 4, 7]

columns: portions of 2D-arrays that are depicted vertically. These are the elements of each array with the same index. [Chapter 10]

comma-separated variables (CSV): a text file format where each line represents a row of data and the values within each row are separated by commas [Chapter 9]

commenting: including portions of the program that do not affect execution but are rather used to make notes for the programmer or for the user [Chapter 3]

compile-time error: a basic programming error identified by the interpreter during compiling [Chapter 3]
compiling: translating a programming language into binary code [Chapter 3]
compound condition: a complicated condition that includes at least one boolean operator [Chapter 5]
Computer Science: different aspects of computing, usually development [Chapter 3]
concatenation operator: a plus sign ("+") used between two strings indicating that the two string values be outputted next to each other [Chapter 3]
condition: the portion of a conditional statement that has a boolean result and determines whether the statement happens [Chapter 5]
conditional statement: a statement that is executed only if some other condition is met [Chapter 5]
constructor: a method in an object class used to build the object [Chapter 7]

D

data set: a collection of values or information that can be organized and analyzed for various purposes. It can include numbers, text, images, or any type of data [Chapter 9]
decrement operator (--): a symbol used after a variable to decrease its value by 1 [Chapter 3]
delimiter: a character or sequence of characters that marks the boundary between separate pieces of data. It helps to structure and organize information by indicating where one piece of data ends and the next begins. Common delimiters include commas (,) and semicolons (;) [Chapter 9]
double: a primitive data type representing numbers that can include decimals [Chapter 3]
driver class: a class that is created to control a larger program [Chapter 7]
dynamically sized: having the ability to change length and to insert and remove elements [Chapter 9]

E

enhanced-`for` loop: a `for` loop with a simplified call used to traverse an array [Chapter 8]
escape sequence: a small piece of coding beginning with a backslash to indicate special characters [Chapter 3]

F

fields: see **instance variables**
flow control: indication of which lines of programming should be executed in which conditions [Chapter 5]
`for` loop: a loop with not only a condition but also an initializer and incrementer to control the truth value of the condition [Chapter 6]
full: all values of the array are assigned [Chapter 8]

H

header: the "title" of a method used to indicate its overall function [Chapter 7]

I

identifiers: names given to indicate data stored in memory [Chapter 3]
`if`: a reserved word in Java indicating the condition by which a statement will be executed [Chapter 5]
immutable: having no mutator methods [Chapter 4]
increment operator (++): a symbol used after a variable to increase its value by 1 [Chapter 3]
index numbers: integers, beginning with 0, used to indicate the order of the elements of an array [Chapter 8]
infinite loop: a programming error in which a loop never terminates because the condition is never false [Chapter 6]
initializer list: known values used to assign the initial values of all the elements of an array [Chapter 8]
in-line comments (short comments): comments preceded by two forward slashes ("//") indicating that the rest of the line of text will not be executed [Chapter 3]

insertion sort: a sorting algorithm in which smaller elements are inserted before larger elements [Chapter 8]
instance variables (fields): variables, listed immediately after the class heading, of an object that are accessible by all of the object's methods [Chapter 7]
integer: a primitive data type representing positive numbers, negative numbers, and 0 with no fractions or decimals [Chapter 3]
interpreter: the part of the developer environment that enables the computer to understand the Java code [Chapter 3]

L

list object: an object form of an array with a dynamic size that can store multiple types of data
logical error: an error that lies in the desired output/purpose of the program rather than in the syntax of the code itself [Chapter 3]
long comments: comments that use ("/*") to indicate the beginning of the comment and ("*/") to indicate the end [Chapter 3]
loop: a portion of code that is to be executed repeatedly [Chapter 6]

M

merge sort: a sorting algorithm in which an array is divided and each half of the array is sorted and later merged into one array [Chapter 8]
method: a group of code that performs a specific task [Chapter 7]
mutator methods: methods used to change the value of a data field [Chapter 7]

N

`null`: the default value of an object that has not yet been assigned a value [Chapter 8]
`NullPointerException`: a run-time error caused by calling an object with a `null` value [Chapter 8]

O

object: an entity or data type created in Java; an instance of a class [Chapter 4]
object class: a class that houses the "guts" of the methods that the driver class calls [Chapter 7]
object reference variable: a variable name [Chapter 7]
overloading: using two methods with the same name but with different numbers and/or types of parameters [Chapter 7]

P

parameters: what types of variables, if any, will be inputted to a method [Chapter 7]
persist: when the data in a file continues to exist, even when the program is not executing [Chapter 9]
planning: outlining the steps of a proposed program [Chapter 7]
postcondition: a comment that is intended to guarantee the user calling the method of a result that occurs as the result of calling the method [Chapter 7]
precedence: the order in which Java will execute mathematical operations, starting with parentheses, followed by multiplication and division from left to right, followed by addition and subtraction from left to right [Chapter 3]
precondition: a comment that is intended to inform the user more about the condition of the method and guarantees it to be true [Chapter 7]
primitive data: the four most basic types of data in Java (`int`, `double`, `boolean`, and `char`) [Chapter 3]
programming style: a particular approach to using a programming language [Chapter 3]

R

recursion: a flow control structure in which a method calls itself [Chapters 8, 11]
recursive call: the command in which a method calls itself [Chapter 11]
reference: a link created to an object when it is passed to another class and, as a result, received through a parameter [Chapter 7]
return type: what type of data, if any, will be outputted by a method after its commands are executed [Chapter 7]
rows: the portions of 2D-arrays that are depicted horizontally and can be seen as their own arrays [Chapter 10]
run-time error: an error that occurs in the execution of the program [Chapter 3]

S

search algorithms: methods of finding a particular element in an array [Chapter 8]
selection sort: a sorting algorithm in which the lowest remaining element is swapped with the element at the lowest unsorted index [Chapter 8]
sequential search: a search of all the elements of an array in order until the desired element is found [Chapter 8]
short-circuit: a process by which the second condition of an *and* or *or* statement can be skipped when the first statement is enough to determine the truth value of the whole statement [Chapters 5, 6]
short comments: see **in-line comments**
sorting algorithms: methods of ordering the elements within an array [Chapter 8]
source code: a .java file that defines a program's actions and functions [Chapter 7]
span: a somewhat dated word that means using a loop to use or change all elements of an array. Now referred to as **traverse**.
`static`: when a program is running and there is only one instance of something. A static object is unique: a static variable is allocated to the memory only once (when the class loads). [Chapter 7]
static method: a non-constructor method that is designed to access and/or modify a static variable [Chapter 7]
static variable: an attribute that is shared among all instances of a class [Chapter 7]
string: see **string literal**
string literal (string): one or more characters combined in a single unit [Chapter 3]
strongly typed: characteristic of a language in which a variable will always keep its type until it is reassigned [Chapter 3]

T

traverse: use a loop to use or change all elements of an array [Chapter 8]
trace table: a table used to trace possibilities in conditionals [Chapter 5]
truth value: the indication of whether a statement is true or false [Chapter 5]
two-dimensional array (2D array): an array in two dimensions, with index numbers assigned independently to each row and column location [Chapter 10]
typed `ArrayList`: an `ArrayList` that allows only one type of data to be stored [Chapter 9]

V

variable: an identifier associated with a particular value [Chapter 3]

W

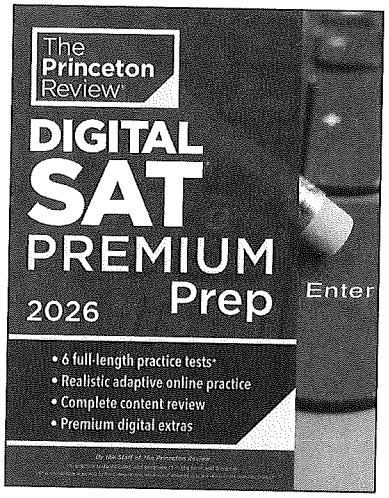
`while` loop: a loop that cycles again and again while a condition is true [Chapter 6]
white space: empty space intended to enhance readability [Chapters 3, 9]

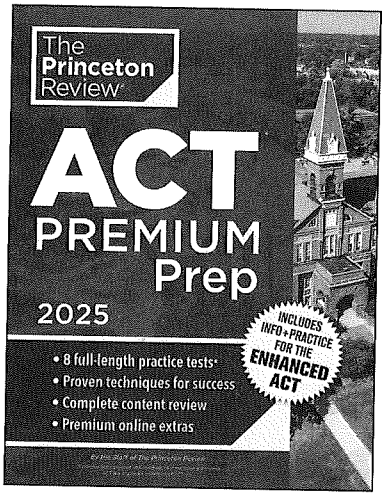
NOTES

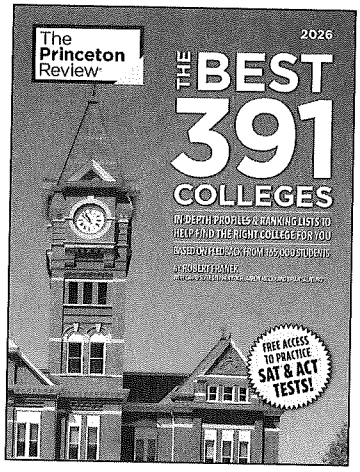
The Princeton Review®

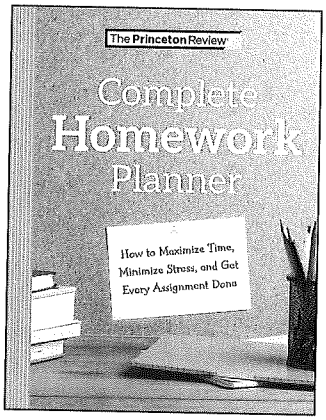
TURN YOUR COLLEGE DREAMS INTO REALITY!

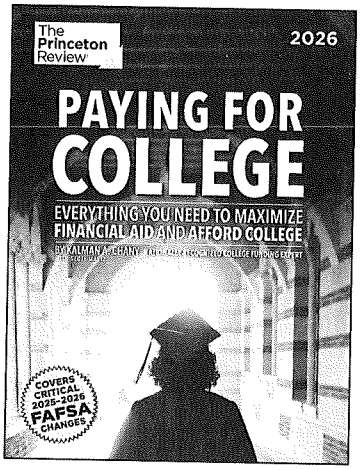
From acing tests to picking the perfect school, The Princeton Review has proven resources to help students like you navigate the college admissions process.

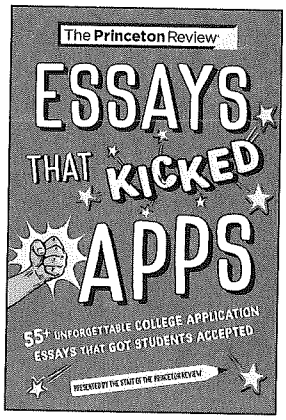












Visit PrincetonReviewBooks.com to browse all of our products!

1634b